Etapa 01 – Modelado de datos y diseño del proyecto

Sistema de Gestión de Pedidos y Reseñas de Restaurantes · MongoDB (Atlas) · CC3089 Base de Datos 2 · Semestre I 2026

Este documento es la guía de diseño y de calificación. La implementación del backend seguirá las decisiones aquí definidas.

Índice detallado

1. Descripción del problema, alcance y supuestos
   - 1.1 Problema
   - 1.2 Alcance
   - 1.3 Supuestos
2. Modelo de datos detallado
   - 2.1 Colecciones y responsabilidad
   - 2.2 Colección `restaurantes`
   - 2.3 Colección `usuarios`
   - 2.4 Colección `articulos_menu`
   - 2.5 Colección `ordenes`
   - 2.6 Colección `reseñas`
   - 2.7 Diagrama de relaciones documentales
3. Justificación embedding vs referencing
   - 3.1 Documentos embebidos
   - 3.2 Referencias

3.3 Manejo de arrays

4. Transacciones y consultas multi-documento

4.1 Caso: Crear orden y actualizar contadores

5. Aggregation pipelines

5.1 Agregaciones simples

5.2 Pipelines complejos

6. Índices y análisis con explain()

6.1 Índices por tipo y colección

6.2 Validación de impacto con explain()

7. Propuesta de shard key y escalabilidad

7.1 Shard keys propuestas

7.2 Análisis de escalabilidad

8. Checklist de rúbrica para implementación

---

1. Descripción del problema, alcance y supuestos

1.1 Problema

Se requiere un sistema que permita a usuarios consultar restaurantes, realizar pedidos y gestionar reseñas. Los datos deben persistirse en MongoDB (Atlas), con un backend que exponga operaciones CRUD, consultas multi-colección, agregaciones, manejo de archivos (GridFS) y uso explícito de índices. La base de datos debe estar configurada para rechazar consultas que no utilicen índices.

1.2 Alcance

- Backend funcional (lenguaje a elección del equipo) que se conecte a MongoDB Atlas.
- Cinco colecciones: `restaurantes`, `usuarios`, `articulos_menu`, `ordenes`, `reseñas`.

- Operaciones CRUD (crear uno/varios, leer con lookups/filtros/proyecciones/ordenamiento/skip/límite, actualizar uno/varios, eliminar uno/varios).
- Uso justificado de documentos embebidos y referencias.
- Al menos una transacción o consulta multi-documento.
- Aggregation Framework para consultas analíticas (simples y pipelines complejos).
- Manejo de arrays (`$push`, `$pull`, `$addToSet`, etc.).
- GridFS y al menos una colección con ≥50.000 documentos iniciales (p. ej. `articulos_menu` con datos semilla).
- Al menos 4 tipos de índices (simple, compuesto, multikey, geoespacial, texto) repartidos en las colecciones, con validación mediante `explain()`.
- Interfaz de uso: menú por consola y/o frontend (extras según rúbrica).

1.3 Supuestos

- Los usuarios deben estar registrados antes de realizar pedidos o reseñas.
- Cada orden pertenece a un único restaurante y a un único usuario.
- Las reseñas pueden asociarse a un restaurante y opcionalmente a una orden concreta.
- Los artículos del menú pertenecen a un restaurante; la colección `articulos_menu` puede contener 50.000+ documentos (múltiples restaurantes y muchos ítems).
- Se utilizará MongoDB Atlas; la aplicación asumirá una URI de conexión por variable de entorno.
- Las coordenadas de restaurantes son en formato GeoJSON (Point) para índices geoespaciales.

---

2. Modelo de datos detallado

2.1 Colecciones y responsabilidad

Colección	Responsabilidad
`restaurantes`	Datos del restaurante, dirección, ubicación geoespacial, nombre, horarios.
`usuarios`	Identidad del usuario (email, nombre); quien realiza pedidos y reseñas.
`articulos_menu`	Ítems del menú por restaurante (nombre, precio, categoría). Candidata a ≥50k docs.

Colección	Responsabilidad
`ordenes`	Pedidos: ítems embebidos, estado, monto total, referencias a usuario y restaurante.
`reseñas`	Reseñas con calificación y texto; referencias a restaurante, usuario y opcionalmente orden.

2.2 Colección `restaurantes`

Campos y tipos:

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador único.
`nombre`	String	Nombre del restaurante.
`descripcion`	String	Descripción breve (búsqueda por texto).
`direccion`	Embedded	Objeto: `calle`, `ciudad`, `pais`.
`ubicacion`	GeoJSON Point	`{ type: "Point", coordinates: [lng, lat] }` para índices geoespaciales.
`horario`	String	Ej. "09:00–22:00".
`activo`	Boolean	Si está disponible para pedidos.
`createdAt`	Date	Fecha de alta.

Ejemplo de documento:

```
{
  "_id": ObjectId("..."),
  "nombre": "La Cocina Verde",
  "descripcion": "Comida saludable y opciones veganas",
  "direccion": { "calle": "Av. Principal 100", "ciudad": "Guatemala", "pais":
"Guatemala" },
  "ubicacion": { "type": "Point", "coordinates": [-90.5133, 14.6349] },
  "horario": "09:00–22:00",
  "activo": true,
```

```
  "createdAt": ISODate("2026-01-15T10:00:00Z")
}
```

2.3 Colección `usuarios`

Campos y tipos:

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador único.
`email`	String	Único; índice simple.
`nombre`	String	Nombre del usuario.
`totalOrdenes`	Number	Contador actualizado en transacción al crear orden.
`createdAt`	Date	Fecha de registro.

Ejemplo de documento:

```
{
  "_id": ObjectId("..."),
  "email": "maria@ejemplo.com",
  "nombre": "María López",
  "totalOrdenes": 5,
  "createdAt": ISODate("2026-01-10T08:00:00Z")
}
```

2.4 Colección `articulos_menu`

Campos y tipos:

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador único.
`restauranteId`	ObjectId	Referencia a `restaurantes`.
`nombre`	String	Nombre del platillo.
`descripcion`	String	Para búsqueda por texto.

Campo	Tipo	Descripción
`categoria`	String	Ej. "Entrada", "Plato fuerte", "Postre".
`precio`	Number	Precio unitario.
`disponible`	Boolean	Si está en menú.
`createdAt`	Date	Fecha de alta.

Esta colección se poblará con ≥50.000 documentos (datos semilla o generados) para cumplir el requisito de la rúbrica.

2.5 Colección `ordenes`

Campos y tipos:

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador único.
`userId`	ObjectId	Referencia a `usuarios`.
`restauranteId`	ObjectId	Referencia a `restaurantes`.
`items`	Array of Embedded	Lista de ítems: `articuloId`, `nombre`, `cantidad`, `precioUnitario`, `subtotal`.
`montoTotal`	Number	Suma de subtotales.
`estado`	String	Ej. "pendiente", "confirmado", "entregado", "cancelado".
`fecha`	Date	Fecha del pedido.
`createdAt`	Date	Timestamp de creación.

Ejemplo de documento (con ítems embebidos):

```
{
  "_id": ObjectId("..."),
  "userId": ObjectId("..."),
  "restauranteId": ObjectId("..."),
  "items": [
    { "articuloId": ObjectId("..."), "nombre": "Ensalada César", "cantidad": 2,
```

```
"precioUnitario": 45.00, "subtotal": 90.00 },
    { "articuloId": ObjectId("..."), "nombre": "Pasta", "cantidad": 1,
"precioUnitario": 75.00, "subtotal": 75.00 }
  ],
  "montoTotal": 165.00,
  "estado": "confirmado",
  "fecha": ISODate("2026-02-20T12:30:00Z"),
  "createdAt": ISODate("2026-02-20T12:30:00Z")
}
```

2.6 Colección `reseñas`

Campos y tipos:

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador único.
`restauranteId`	ObjectId	Referencia a `restaurantes`.
`userId`	ObjectId	Referencia a `usuarios`.
`ordenId`	ObjectId (opcional)	Referencia a `ordenes` si la reseña es sobre un pedido.
`calificacion`	Number	1–5.
`comentario`	String	Texto de la reseña.
`createdAt`	Date	Fecha de la reseña.

2.7 Diagrama de relaciones documentales

```
  restaurantes (1) ←——— (N) articulos_menu  [articulos_menu.restauranteId]
  restaurantes (1) ←——— (N) ordenes         [ordenes.restauranteId]
  restaurantes (1) ←——— (N) reseñas         [reseñas.restauranteId]
  usuarios (1)     ←——— (N) ordenes         [ordenes.userId]
  usuarios (1)     ←——— (N) reseñas         [reseñas.userId]
  ordenes (1)      ←——— (N) reseñas         [reseñas.ordenId, opcional]
  ordenes: items[] = documentos embebidos (nombre, cantidad, precioUnitario,
subtotal)
```

3. Justificación embedding vs referencing

3.1 Documentos embebidos

- **Orden → items:** Los ítems de cada pedido se guardan embebidos en el documento de la orden porque (1) siempre se leen y escriben junto con la orden, (2) el tamaño es acotado (cantidad razonable de ítems por orden), (3) no se consultan solos en otras colecciones. Permite proyecciones y agregaciones sobre `ordenes.items` (ej. platillos más vendidos con `$unwind`).
- **Restaurante → direccion:** La dirección es un objeto pequeño y estable; siempre se muestra con el restaurante. Evita un lookup extra.
- **Restaurante → ubicacion (GeoJSON):** Embebido como parte del documento para índices geoespaciales en la misma colección.

3.2 Referencias

- **ordenes.userId, ordenes.restauranteId:** Se usan referencias para poder hacer lookups y obtener datos actualizados de usuario y restaurante (nombre, email, dirección) sin duplicar datos que pueden cambiar.
- **articulos_menu.restauranteId:** Referencia para listar menú por restaurante y para agregaciones por restaurante.
- **reseñas.restauranteId, userId, ordenId:** Referencias para consultas multi-colección (restaurantes mejor calificados, reseñas por usuario, reseñas por orden).

3.3 Manejo de arrays (decisión para implementación)

Para cumplir la rúbrica de manejo de arrays (`$push`, `$pull`, `$addToSet`):

- **ordenes.items:** Añadir ítem a una orden existente con `$push` a `items`; quitar ítem con `$pull` (por `articuloId` o por condición). Recalcular `montoTotal` en la misma actualización o en un segundo paso.
- **Opcional en restaurantes:** Si se agrega un array `tags` (ej. ["vegano", "rápido"]), usar `$addToSet` para añadir tags sin duplicar y `$pull` para quitar.

---

4. Transacciones y consultas multi-documento

4.1 Caso: Crear orden y actualizar contadores

Objetivo: Garantizar que al crear una nueva orden se actualice atómicamente el contador `totalOrdenes` del usuario.

Pasos dentro de la transacción:

1. Iniciar transacción (session).
2. Insertar un documento en `ordenes` (con `userId`, `restauranteId`, `items` embebidos, `montoTotal`, `estado`, `fecha`).
3. Actualizar el documento en `usuarios` correspondiente a `userId`: `$inc: { totalOrdenes: 1 }`.
4. Commit de la transacción.

Implementación: Usar `startTransaction()` / `commitTransaction()` (o equivalente del driver) y en caso de error `abortTransaction()`. Todas las operaciones de la transacción usarán la misma session.

---

5. Aggregation pipelines

5.1 Agregaciones simples

Se implementarán en el backend usando `countDocuments`, `distinct` o pipelines cortos de una etapa:

- **Conteo por estado de orden:** `ordenes.countDocuments({ estado: "pendiente" })` (o por restaurante/usuario).
- **Distinct de categorías de menú:** `articulos_menu.distinct("categoria", { restauranteId: id })`.
- **Conteo de reseñas por restaurante:** `reseñas.countDocuments({ restauranteId: id })` o `$group` por `restauranteId` con `$sum: 1`.

5.2 Pipelines complejos

Pipeline 1: Restaurantes mejor calificados

Objetivo: listar restaurantes ordenados por calificación promedio de sus reseñas (con lookup de datos del restaurante).

Colección principal: `reseñas`.

- `$match`: opcional filtro por fecha o por `restauranteId`.
- `$group`: `_id: "$restauranteId"`, `promedioCalificacion: { $avg: "$calificacion" }`, `totalReseñas: { $sum: 1 }`.
- `$sort`: `{ promedioCalificacion: -1 }`.
- `$limit`: N (ej. 10).
- `$lookup`: desde `restaurantes`, `localField: "_id"`, `foreignField: "_id"`, `as: "restaurante"`.
- `$unwind`: `"$restaurante"`.
- `$project`: campos deseados (nombre, dirección, promedioCalificacion, totalReseñas).

Pipeline 2: Platillos más vendidos (por nombre o por articuloId)

Objetivo: reporte analítico de ítems más pedidos (usando array embebido `ordenes.items`).

Colección principal: `ordenes`.

- `$match`: opcional por `restauranteId` o rango de fechas.
- `$unwind`: `"$items"`.
- `$group`: `_id: "$items.nombre"` (o `items.articuloId`), `cantidadVendida: { $sum: "$items.cantidad" }`, `ventasTotales: { $sum: "$items.subtotal" }`.
- `$sort`: `{ cantidadVendida: -1 }`.
- `$limit`: N.
- `$project`: proyección para respuesta.

Pipeline 3: Órdenes con datos de usuario y restaurante (lectura con lookups)

Objetivo: listar órdenes con filtros, proyecciones, ordenamiento, skip y límite, enriqueciendo con datos de usuario y restaurante.

Colección principal: `ordenes`.

- `$match`: filtros (estado, restauranteId, userId, rango de fechas).
- `$sort`: `{ fecha: -1 }` (u otro campo).
- `$skip`: paginación.
- `$limit`: tamaño de página.

- `$lookup`: `usuarios`, `localField: "userId"`, `foreignField: "_id"`, `as: "usuario"`.
- `$lookup`: `restaurantes`, `localField: "restauranteId"`, `foreignField: "_id"`, `as: "restaurante"`.
- `$unwind`: usuario y restaurante (con `preserveNullAndEmptyArrays` si se desea).
- `$project`: solo campos necesarios (evitar traer documentos completos).

---

6. Índices y análisis con explain()

6.1 Índices por tipo y colección

La base se configurará para rechazar consultas que no usen índices (`notablescan` o equivalente). Cada consulta del backend debe utilizar al menos un índice.

Tipo	Colección	Definición	Justificación / consulta que lo usa
Simple	`usuarios`	`{ email: 1 }` (unique)	Login y búsqueda por email; evitar duplicados.
Compuesto	`ordenes`	`{ restauranteId: 1, fecha: -1 }`	Listar órdenes de un restaurante ordenadas por fecha; pipelines con $match + $sort.
Compuesto	`ordenes`	`{ userId: 1, estado: 1 }`	Órdenes de un usuario por estado.
Multikey	`ordenes`	`{ "items.nombre": 1 }` o `{ "items.articuloId": 1 }`	Agregaciones que filtran o agrupan por ítem (platillos más vendidos).
Geoespacial	`restaurantes`	`{ ubicacion: "2dsphere" }`	Restaurantes cercanos: `$geoNear` o `$match` con `$geoWithin` / `$near`.

Tipo	Colección	Definición	Justificación / consulta que lo usa
Texto	`restaurantes`	`{ nombre: "text", descripcion: "text" }`	Búsqueda por texto en nombre y descripción.
Compuesto	`articulos_menu`	`{ restauranteId: 1, categoria: 1 }`	Menú por restaurante y categoría.
Compuesto	`reseñas`	`{ restauranteId: 1, calificacion: -1 }`	Restaurantes mejor calificados; pipelines sobre reseñas.

Se cumple con al menos: simple (usuarios.email), compuesto (ordenes, reseñas, articulos_menu), multikey (ordenes.items), geoespacial (restaurantes.ubicacion), texto (restaurantes).

6.2 Validación de impacto con explain()

Para cada tipo de consulta crítica (las que usan los índices anteriores), en implementación se ejecutará `explain("executionStats")` y se documentará o verificará:

- Que el `winningPlan.inputStage.stage` sea `IXSCAN` (uso de índice) y no `COLLSCAN`.
- Que `totalDocsExamined` sea cercano a `nReturned` cuando se espera pocos documentos, indicando buen uso del índice.

Ejemplo de verificación: consulta `ordenes.find({ restauranteId: id }).sort({ fecha: -1 })` debe usar el índice `{ restauranteId: 1, fecha: -1 }` y mostrar `IXSCAN` en el explain.

---

7. Propuesta de shard key y escalabilidad

7.1 Shard keys propuestas

- **ordenes:** `{ restauranteId: 1, fecha: 1 }`. Las consultas frecuentes filtran por restaurante; la fecha permite distribuir documentos dentro de cada restaurante y evita hotspots si un solo restaurante crece mucho (en ese caso se podría considerar hashed `restauranteId`).

- **restaurantes:** `{ _id: "hashed" }` o por región si se modela un campo `region`. Para crecimiento uniforme.
- **reseñas:** `{ restauranteId: 1 }`. Las consultas analíticas suelen agrupar por restaurante.
- **articulos_menu:** `{ restauranteId: 1 }`. Menú siempre se consulta por restaurante.
- **usuarios:** `{ _id: "hashed" }` o `{ email: "hashed" }` para distribución uniforme.

7.2 Análisis de escalabilidad

Incluir en el diseño que las consultas que filtren por `restauranteId` (ordenes, reseñas, articulos_menu) serán eficientes en un cluster shardeado porque la shard key está en el filtro. Las consultas que no incluyan la shard key (ej. todas las órdenes de todos los restaurantes) pueden resultar en scatter-gather; para reportes globales se puede usar agregaciones que recorran todos los shards. La propuesta prioriza el caso de uso “por restaurante” que es el más común.